

# pamica: GPU-accelerated Adaptive Mixture Independent Component Analysis in Python with Fortran parity

Seyed Yahya Shirazi<sup>1</sup>, Arnaud Delorme<sup>1,2</sup>, and Scott Makeig<sup>1</sup>

<sup>1</sup> Swartz Center for Computational Neuroscience, Institute for Neural Computation, University of California San Diego, USA <sup>2</sup> Centre de Recherche Cerveau et Cognition (CerCo), CNRS, University of Toulouse, France ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

## Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

## Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

## License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

## Summary

Independent Component Analysis (ICA) separates electroencephalographic and magnetoencephalographic (EEG/MEG) recordings into maximally independent sources that isolate brain, muscle, and artifact activities for downstream analysis ([Iversen & Makeig, 2019](#); [Makeig et al., 1995](#); [Vigário et al., 1997](#)). Adaptive Mixture ICA (AMICA) ([Palmer et al., 2012](#)) models each source with a flexible, self-adjusting probability density and lets several ICA models coexist, one per segment of a recording. Among the algorithms benchmarked by Delorme et al. (2012) it recovers the least dependent and most dipolar decompositions, and therefore the most physiologically interpretable ones. Its reference implementation, by Jason Palmer, is a Fortran program distributed as a compiled binary callable from MATLAB/EEGLAB: awkward to install, restricted to the central processing unit (CPU), and unusable from Python.

pamica reproduces the reference Fortran results within numerical tolerance while running on the CPU, NVIDIA graphics processing units (GPUs, via CUDA), and Apple GPUs (through Apple's MLX array framework ([Hannun et al., 2023](#))). It is a complete reimplement on PyTorch ([Paszke et al., 2019](#)), NumPy ([Harris et al., 2020](#)), and SciPy ([Virtanen et al., 2020](#)) rather than a wrapper around the binary, and exposes a scikit-learn-style estimator under a BSD-3-Clause license. It writes the format EEGLAB's AMICA loader reads, so established MATLAB tooling consumes its results unchanged. The software is at <https://github.com/sccn/pAMICA> (archived at doi:10.5281/zenodo.21312148).

## Statement of need

AMICA decompositions are well suited to equivalent-dipole source localization and automated component classification ([Pion-Tonachini et al., 2019](#)). Yet the reference implementation is MATLAB-only Fortran, an increasing obstacle as neuroimaging analysis moves toward Python, for example MNE-Python ([Gramfort et al., 2013](#)): modern pipelines need an AMICA that runs natively in Python and on a GPU, and that is validated to reproduce the Fortran reference numerically.

General-purpose Python ICA implementations do not fill this gap. scikit-learn and MNE-Python provide FastICA ([Hyvärinen & Oja, 2000](#)) and Infomax ([Bell & Sejnowski, 1995](#); [Lee et al., 1999](#)), while Picard ([Ablin et al., 2018](#)) offers faster-converging maximum-likelihood ICA; none implement AMICA's mixture of models, adaptive generalized-Gaussian densities, or Newton updates, so none can reproduce its decompositions. pamica is for analysts who want AMICA-quality decompositions in Python, for anyone with a GPU who wants faster runs than the CPU-only binary, and for methodologists who need a transparent reference to build on.

41 Validation to date is on EEG; the algorithm is modality-agnostic but MEG is untested.

## 42 State of the field

43 pamica complements rather than replaces the reference Fortran AMICA used with EEGLAB  
44 (Delorme & Makeig, 2004): it keeps the same output format, adds a Python API with GPU  
45 support, and can run the reference Fortran itself through a bundled dependency-free native  
46 build. Three other Python AMICA reimplementations have appeared as of August 2026  
47 (Esmaili, 2025; Herforth, 2026; Huberty, 2025), oriented toward MNE-Python; pamica adds  
48 a scikit-learn-style array API, output in EEGLAB's exact on-disk layout, an MLX backend,  
49 and an optional MNE-Python wrapper. Its Fortran-parity validation goes further than theirs:  
50 score functions exact to floating-point resolution against the literal Fortran expressions, and a  
51 distributional framework for the non-identifiable multi-model case.

## 52 Software design

53 The governing decision was to treat numerical parity with Palmer's Fortran, rather than  
54 convergence to some independent solution, as the definition of correctness. The rest follows  
55 from it. Wrapping the binary would have secured parity for free but inherited its CPU-only,  
56 MATLAB-facing design; reimplementing put parity at risk. pamica does both, porting the  
57 algorithm natively and also shipping the reference binary as a runnable engine, so users  
58 can reproduce the parity claims on their own data and hardware. For the same reason the  
59 port follows the reference's natural-gradient (Amari, 1998) expectation-maximization (EM)  
60 formulation rather than an automatic-differentiation optimizer: an Adam/autograd backend  
61 was written early and then deleted, because it converged to different optima and made the  
62 name "AMICA" ambiguous. The port covers exact-EM mixture updates, a positive-definite  
63 Newton step (Palmer et al., 2008), symmetric sphering, the five source-density families, a  
64 mixture of ICA models, component sharing, and the mutual-information metrics used to score  
65 separation quality (Frank et al., 2023).

66 Three array backends (PyTorch, MLX, NumPy) implement the same algorithm behind a  
67 common estimator API, with the reference Fortran a fourth. The duplication is deliberate:  
68 NumPy stays readable as an executable specification, and MLX exists because PyTorch's Metal  
69 backend is slower than the CPU on Apple hardware. Double precision is the default because  
70 parity demands it; single precision is available for Apple GPUs, which have no float64.

## 71 Validation

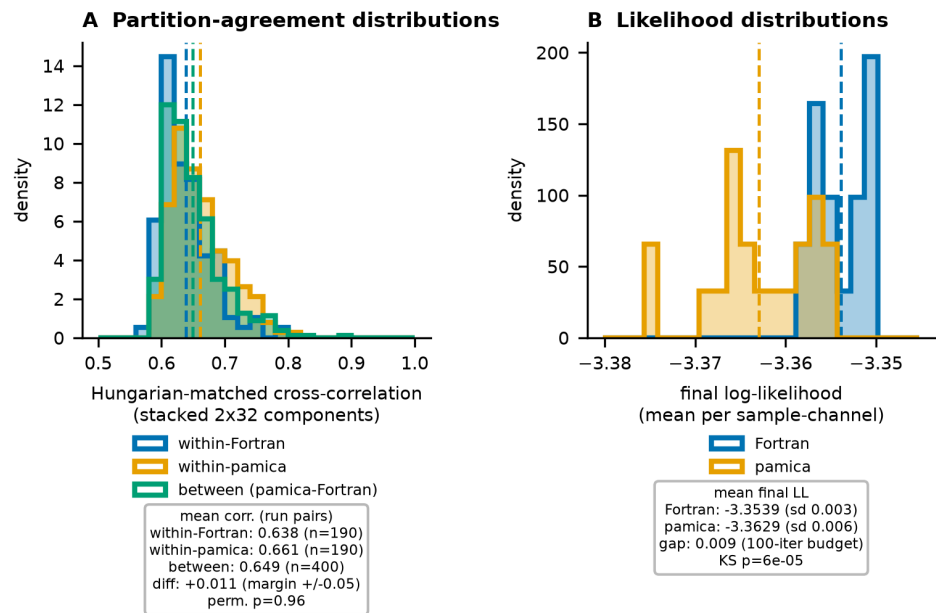
72 Parity is measured two ways: by Hungarian-matched component correlation, and by the  
73 Amari distance (Amari et al., 1996), a relabeling- and scale-invariant metric that needs no  
74 assignment step. Both implementations ran AMICA's default 2000 iterations with Newton  
75 disabled (pamica's own default), to isolate the algorithm from initialization. With Newton  
76 enabled and independent seeds, some of the weakest components settle into a different basin  
77 of equal or higher likelihood: on one seed of three this affected ten of seventy components,  
78 while the other two matched the reference at  $\sim 0.99$ . A matched initialization restores  $\sim 0.997$ ,  
79 so this is a property of the initialization, not a parity defect. The single-model comparison uses  
80 a well-determined external recording (OpenNeuro ds002718,  $k \approx 153$ , where  $k$  = frames over  
81 squared channel count (Frank et al., 2025)) alongside the bundled 32-channel sample ( $k \approx 30$ ).  
82 A mixture of ICA models is not partition-identifiable, so exact partition parity is the wrong  
83 bar there; it is judged instead by whether the implementations sample a similar distribution of  
84 solutions, over ensembles of 20 runs each (Figure 1). A permutation test finds no evidence  
85 that cross-implementation agreement is worse than Fortran's own run-to-run agreement.

**Table 1:** Parity of pamica with the Fortran reference. Multi-model rows are over 20-run ensembles (190 within-, 400 cross-implementation pairs); sd is the standard deviation, given where computed. The final row is the one metric on which the ensembles differ significantly, and it is convergence speed rather than optimum quality: pamica reaches Fortran's mean by 200 iterations and passes it by 300.

| Regime | Metric (dataset)                            | Result (mean)                                                       |
|--------|---------------------------------------------|---------------------------------------------------------------------|
| Single | Log-likelihood gap (ds002718)               | within $\sim 0.0005$ of $-3.6993$                                   |
| Single | Component correlation (ds002718)            | 0.998                                                               |
| Single | Amari distance (bundled)                    | 0.006                                                               |
| Single | Score functions, sufficient statistics      | exact, $\sim 10^{-15}$                                              |
| Multi  | Correlation, one run: cross; within-Fortran | 0.65; 0.64 (sd 0.05)                                                |
| Multi  | Amari, one run: cross; within-Fortran       | 0.163; 0.174 (sd 0.02)                                              |
| Multi  | Ensemble agreement, cross — within-Fortran  | correlation $+0.011$ ( $p = 0.96$ ); Amari $-0.011$ ( $p > 0.999$ ) |
| Multi  | Ensemble log-likelihood: Fortran; pamica    | $-3.3539$ ; $-3.3629$ (Kolmogorov-Smirnov $p = 6 \times 10^{-5}$ )  |

#### Multi-model AMICA (n\_models=2): pamica vs Fortran ensembles, real sample EEG

Dashed vertical lines mark each distribution's mean.



**Figure 1:** Multi-model ensemble partition-correlation (A) and log-likelihood (B) distributions, 20 pamica and 20 Fortran fits of the sample EEG; dashed lines mark each mean. A's three distributions overlap, so the single-run correlation reflects intrinsic run-to-run spread, not a gap to the reference. B's separation is Table 1's 0.009 gap on a  $\sim 0.035$  axis.

86 All backends converge to the same single-model log-likelihood on real EEG (maximum pairwise

87 difference  $\sim 0.003$ ), and single precision matches double to four or five significant digits on  
88 that log-likelihood. Component-level float32 agreement is not yet characterized at a matched  
89 iteration budget, so float64 remains the default for parity work, and double-precision CUDA is  
90 the reproducible NVIDIA path.

91 On real 70-channel EEG, per-iteration cost is 25 ms for MLX on an Apple GPU, 39 ms for  
92 double-precision CUDA on an RTX 4090, and 30 ms for native Fortran on a 24-core i9-13900K,  
93 against 193 ms for PyTorch on an Apple-Silicon CPU and 255 ms for PyTorch-Metal, which  
94 never beats the CPU it runs beside. Full tables, the data-size sweep, and reproduction  
95 commands are in the [documentation](#); the correctness harness never uses synthetic data.

## 96 Research impact statement

97 pamica was first released in July 2026, so its case rests on readiness and early use rather than  
98 accumulated citations.

99 Because pamica writes the reference binary's output format, existing EEGLAB analyses read  
100 its decompositions unchanged, so a lab can adopt Python without re-tooling everything  
101 downstream. It also redistributes the reference Fortran as a dependency-free build for macOS,  
102 Linux, and Windows.

103 Seven releases have been published, four of them to the Python Package Index (513 downloads  
104 in the month before submission), with an archived Zenodo record. One user outside the  
105 author group reported a bug from their own 236-channel, 8.3-million-sample decomposition  
106 (sccn/pAMICA issue 207); another, the author of a competing reimplementation, raised  
107 completeness and packaging questions (issue 206). MNE-Python is publicly weighing which  
108 AMICA implementation to adopt (mne-tools/mne-python issue 13819). Integration into this  
109 Center's Python preprocessing and the NEMAR archive is underway, not complete. The  
110 harness, sample data, and reproduction commands ship with the package, so a third party can  
111 re-run Table 1.

## 112 AI usage disclosure

113 Generative AI was used in this project, disclosed here under the journal's policy.

114 **Tools.** Anthropic's Claude models (Sonnet and Opus families), through the Claude Code  
115 command-line assistant. The instructions given to them are public in the repository (AGENTS.md,  
116 CLAUDE.md, .rules/).

117 **Scope.** The source code (translating the reference Fortran into Python, refactoring, scaffolding  
118 tests), the documentation, and the drafting and copy-editing of this manuscript.

119 **Human oversight.** The authors made the design decisions and take responsibility for the  
120 result. Parity as the correctness criterion, the natural-gradient EM formulation, the backend  
121 architecture, the distributional treatment of the multi-model case, and the acceptance thresholds  
122 were chosen by the authors, not by a model. Every AI-assisted change was reviewed by a  
123 human before merge and run through the parity harness, which scores output against the  
124 reference binary on real recordings rather than generated fixtures. The reported numbers  
125 came from running the software and were checked against their run records, as was every  
126 bibliographic entry.

## 127 Acknowledgements

128 We thank Jason Palmer and his advisor Ken Kreutz-Delgado, co-developers of AMICA, for  
129 the reference implementation, and the EEGLAB community for the tools and sample data  
130 used to validate this work. Two authors developed the methods pamica builds on: S.M.

co-developed AMICA (Palmer et al., 2012) and A.D. is a lead developer of EEGLAB (Delorme & Makeig, 2004). This work was supported by The Swartz Foundation (Old Field, NY) to the Swartz Center for Computational Neuroscience and by National Institutes of Health grant R01-NS047293 (to A.D. and S.M.).

## References

- Ablin, P., Cardoso, J.-F., & Gramfort, A. (2018). Faster independent component analysis by preconditioning with hessian approximations. *IEEE Transactions on Signal Processing*, 66(15), 4040–4053. <https://doi.org/10.1109/TSP.2018.2844203>
- Amari, S.-I. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276. <https://doi.org/10.1162/089976698300017746>
- Amari, S.-I., Cichocki, A., & Yang, H. H. (1996). A new learning algorithm for blind signal separation. *Advances in Neural Information Processing Systems*, 8, 757–763.
- Bell, A. J., & Sejnowski, T. J. (1995). An information-maximization approach to blind separation and blind deconvolution. *Neural Computation*, 7(6), 1129–1159. <https://doi.org/10.1162/neco.1995.7.6.1129>
- Delorme, A., & Makeig, S. (2004). EEGLAB: An open source toolbox for analysis of single-trial EEG dynamics including independent component analysis. *Journal of Neuroscience Methods*, 134(1), 9–21. <https://doi.org/10.1016/j.jneumeth.2003.10.009>
- Delorme, A., Palmer, J., Onton, J., Oostenveld, R., & Makeig, S. (2012). Independent EEG sources are dipolar. *PLoS ONE*, 7(2), e30135. <https://doi.org/10.1371/journal.pone.0030135>
- Esmaili, S. (2025). *Amica: Native python implementation of AMICA for MNE-Python and scientific EEG workflows*. <https://github.com/snesmaeli/amica>.
- Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2023). An exploration of optimal parameters for efficient blind source separation of EEG recordings using AMICA. *2023 IEEE 23rd International Conference on Bioinformatics and Bioengineering (BIBE)*, 205–210. <https://doi.org/10.1109/BIBE60311.2023.00040>
- Frank, G., Shirazi, S. Y., Palmer, J., Cauwenberghs, G., Makeig, S., & Delorme, A. (2025). Quantitative exploration of sufficient data quantities for EEG decomposition using AMICA. *2025 12th International IEEE/EMBS Conference on Neural Engineering (NER)*, 532–536. <https://doi.org/10.1109/NER61569.2025.11588993>
- Gramfort, A., Luessi, M., Larson, E., Engemann, D. A., Strohmeier, D., Brodbeck, C., Goj, R., Jas, M., Brooks, T., Parkkonen, L., & Hämäläinen, M. (2013). MEG and EEG data analysis with MNE-Python. *Frontiers in Neuroscience*, 7, 267. <https://doi.org/10.3389/fnins.2013.00267>
- Hannun, A., Digani, J., Katharopoulos, A., & Collobert, R. (2023). *MLX: Efficient and flexible machine learning on apple silicon*. <https://github.com/ml-explore/mlx>.
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., & others. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Herforth, J. (2026). *Pyamica: PyTorch AMICA, adaptive mixture independent component analysis*. <https://github.com/DerAndereJohannes/pyamica>.
- Huberty, S. (2025). *Amica-python: Python implementation of adaptive mixture ICA*. <https://github.com/scott-huberty/amica-python>.
- Hyvärinen, A., & Oja, E. (2000). Independent component analysis: Algorithms and applications.

- 176 *Neural Networks*, 13(4-5), 411–430. [https://doi.org/10.1016/S0893-6080\(00\)00026-5](https://doi.org/10.1016/S0893-6080(00)00026-5)
- 177 Iversen, J. R., & Makeig, S. (2019). MEG/EEG data analysis using EEGLAB. In  
178 *Magnetoencephalography: From signals to dynamic cortical networks* (pp. 391–406).  
179 Springer International Publishing. [https://doi.org/10.1007/978-3-319-62657-4\\_8-1](https://doi.org/10.1007/978-3-319-62657-4_8-1)
- 180 Lee, T.-W., Girolami, M., & Sejnowski, T. J. (1999). Independent component analysis using  
181 an extended infomax algorithm for mixed subgaussian and supergaussian sources. *Neural*  
182 *Computation*, 11(2), 417–441. <https://doi.org/10.1162/089976699300016719>
- 183 Makeig, S., Bell, A. J., Jung, T.-P., & Sejnowski, T. J. (1995). Independent component  
184 analysis of electroencephalographic data. *Advances in Neural Information Processing*  
185 *Systems*, 8, 145–151.
- 186 Palmer, J. A., Kreutz-Delgado, K., & Makeig, S. (2012). *AMICA: An adaptive*  
187 *mixture of independent component analyzers with shared components*. Swartz  
188 Center for Computational Neuroscience, University of California San Diego.  
189 [https://sccn.ucsd.edu/~jason/amica\\_a.pdf](https://sccn.ucsd.edu/~jason/amica_a.pdf)
- 190 Palmer, J. A., Kreutz-Delgado, K., Rao, B. D., & Makeig, S. (2008). Newton method for the  
191 ICA mixture model. *2008 IEEE International Conference on Acoustics, Speech and Signal*  
192 *Processing (ICASSP)*, 1805–1808. <https://doi.org/10.1109/ICASSP.2008.4517982>
- 193 Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin,  
194 Z., Gimelshein, N., Antiga, L., & others. (2019). PyTorch: An imperative style, high-  
195 performance deep learning library. *Advances in Neural Information Processing Systems*, 32,  
196 8024–8035.
- 197 Pion-Tonachini, L., Kreutz-Delgado, K., & Makeig, S. (2019). ICLabel: An automated  
198 electroencephalographic independent component classifier, dataset, and website.  
199 *NeuroImage*, 198, 181–197. <https://doi.org/10.1016/j.neuroimage.2019.05.026>
- 200 Vigário, R., Jousmäki, V., Härmäläinen, M., Hari, R., & Oja, E. (1997). Independent component  
201 analysis for identification of artifacts in magnetoencephalographic recordings. *Advances in*  
202 *Neural Information Processing Systems*, 10, 229–235.
- 203 Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D.,  
204 Burovski, E., Peterson, P., Weckesser, W., Bright, J., & others. (2020). SciPy 1.0:  
205 Fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3),  
206 261–272. <https://doi.org/10.1038/s41592-019-0686-2>